

```
d  
"  
)  
;  
  
}  
}
```

Deine Klasse ist bereits relativ gut für Tests geeignet — aber aktuell gibt es einige Probleme, die automatisierte MariaDB-Tests schwierig machen.

Die wichtigste Regel:

Tests dürfen niemals gegen deine echte Produktionsdatenbank laufen.

Das Problem in deinem aktuellen Code

Du erzeugst Sequelize direkt im Konstruktor:

TypeScript

```
const sequelize = new Sequelize(...)
```

Dadurch kannst du:

- keine Testdatenbank injizieren
- keine InMemory-DB nutzen
- keine isolierten Tests schreiben
- keine parallelen Tests machen

Bessere Architektur

Du solltest `sequelize` von außen übergeben.

Schritt 1: API umbauen

Besser:

TypeScript

```
export class API {  
  
  constructor(private sequelize: Sequelize) {  
  
    this.initModels();  
  
  }  
  
  private initModels() {  
  
    Nutzer.init(...);  
  
  }  
}
```

Schritt 2: Produktions-Sequelize

database.ts

TypeScript

```
import { Sequelize } from 'sequelize';  
  
export const sequelize = new Sequelize(  
  'database',  
  'appuser',  
  'secret123',  
  {  
    host: 'localhost',  
    dialect: 'mariadb',  
    logging: false,  
  }  
);
```

Schritt 3: App starten

TypeScript

```
import { sequelize } from './database.js';
import { API } from './API.js';

const api = new API(sequelize);

await sequelize.sync();
```

Jetzt werden Tests einfach

Beste Lösung für Vitest

Für echte DB-Tests:

Eigene Testdatenbank

database_test

test/setup.ts

TypeScript

```
import { Sequelize } from 'sequelize';

export const testSequelize = new Sequelize(
  'database_test',
  'appuser',
  'secret123',
  {
    host: 'localhost',
    dialect: 'mariadb',
    logging: false,
  }
);
```

API-Test

API.test.ts

TypeScript

```
import { beforeAll, beforeEach, afterAll, describe, expect, test } from 'vitest';

import { API } from '../src/API.js';
import { testSequelize } from './setup.js';

let api: API;

beforeAll(async () => {

    api = new API(testSequelize);

    await testSequelize.sync({ force: true });

});

beforeEach(async () => {

    await testSequelize.truncate({
        cascade: true,
        restartIdentity: true,
    });

});

afterAll(async () => {

    await testSequelize.close();

});

describe('Registrieren', () => {

    test('Nutzer registrieren', async () => {

        const result = await api.registrieren(
            'tim',
            'hash123'
        );

        expect(result).toBe(true);

    });

    test('Doppelter Nutzer schlägt fehl', async () => {

        await api.registrieren(
            'tim',
            'hash123'
        );

        await expect(
            ...
        )
```

```
        api.registrieren(  
            'tim',  
            'hash123'  
        )  
    ).rejects.toThrow();  
  
});  
  
});
```

Wichtiger Fehler in deinem Code

Du benutzt:

TypeScript

```
type: DataTypes.UUIDV4
```

Das ist falsch.

Richtig:

TypeScript

```
type: DataTypes.UUID,  
defaultValue: DataTypes.UUIDV4,
```

Falsch:

TypeScript

```
uuid: new UUIDV4()
```

Richtig:

TypeScript

```
uuid: crypto.randomUUID()
```

oder Sequelize automatisch generieren lassen.

Sehr wichtig

Du verwendest überall:

```
TypeScript
```

```
String
```

Das sollte sein:

```
TypeScript
```

```
string
```

In TypeScript fast immer Primitive:

- string
- number
- boolean

nicht:

- String
- Number
- Boolean

Noch besser: SQLite für Tests

MariaDB-Tests sind langsamer.

Viele Projekte machen:

Produktion

```
TypeScript
```

```
dialect: 'mariadb'
```

Tests

TypeScript

```
dialect: 'sqlite',  
storage: ':memory:'
```

Dann laufen Tests extrem schnell.

Beispiel InMemory-Test

TypeScript

```
const sequelize = new Sequelize({  
  dialect: 'sqlite',  
  storage: ':memory:',  
  logging: false,  
});
```

Das ist die professionelle Standardlösung.

Empfehlung für dein Projekt

Produktion

- MariaDB
- Docker

Tests

- SQLite InMemory
- Vitest
- sequelize.sync({ force: true })

Moderne Struktur


```
src/  
├─ API.ts  
├─ models/  
└─ database.ts  
  
tests/  
├─ setup.ts  
└─ API.test.ts
```

Noch ein Problem

Diese Methode speichert das Sessiontoken vermutlich nicht:

TypeScript

```
entry?.setDataValue("sessiontoken", sessiontoken);
```

Du brauchst:

TypeScript

```
await entry?.save();
```

Professionelles Setup

Ich würde dir empfehlen:

- Vitest
- SQLite InMemory für Tests
- Docker MariaDB für Produktion
- Sequelize Dependency Injection
- beforeEach → Tabellen resettet

Das ist exakt das Setup vieler echter Node.js-Backends.

